

Université Euromed de Fès
Cybersecurity Engineering

TP5 - HTTP Communication

Module : Distributed Applications

Réalisé par :
Mouaad Bouziani

Année Universitaire :
2025-2026

Table des matières

1	Introduction	2
2	Objectifs	2
3	Technologies Utilisées	2
4	Serveur HTTP	2
5	Client HTTP	2
6	Mécanisme de Retry	2
7	Programmation Asynchrone	3
8	Résultats Obtenus	3
9	Conclusion	3

1 Introduction

Les systèmes distribués reposent sur l'échange d'informations entre plusieurs applications connectées à travers un réseau. Le protocole HTTP constitue aujourd'hui l'une des principales méthodes permettant cette communication.

L'objectif de ce travail pratique est de mettre en œuvre un serveur HTTP et un client HTTP afin de comprendre les mécanismes fondamentaux utilisés dans les applications réparties.

2 Objectifs

Les objectifs de ce TP sont les suivants :

- Comprendre l'architecture client-serveur.
- Développer un serveur HTTP en Python.
- Développer un client HTTP.
- Gérer les délais d'attente (timeout).
- Implémenter un mécanisme de retry.
- Découvrir la programmation asynchrone.

3 Technologies Utilisées

- Python 3
- HTTP
- JSON
- urllib
- asyncio

4 Serveur HTTP

Un serveur HTTP a été développé en utilisant la bibliothèque standard Python.

Le serveur contient une collection de ressources stockées en mémoire et fournit les informations sous format JSON.

Afin de simuler des conditions réelles de fonctionnement, une latence réseau aléatoire est ajoutée avant chaque réponse.

5 Client HTTP

Le client permet d'interroger le serveur et de récupérer les ressources disponibles.

Chaque requête est exécutée avec un délai maximal d'attente afin d'éviter les blocages prolongés en cas de ralentissement du serveur.

6 Mécanisme de Retry

Pour améliorer la robustesse du système, un mécanisme de retry a été implémenté.

Lorsqu'une requête échoue, plusieurs tentatives sont effectuées automatiquement.

Une stratégie de backoff exponentiel est utilisée afin d'espacer progressivement les nouvelles tentatives.

7 Programmation Asynchrone

La bibliothèque `asyncio` a été utilisée afin de comparer deux approches :

- Exécution séquentielle.
- Exécution parallèle.

Les résultats montrent que les appels parallèles permettent de réduire considérablement le temps total d'exécution.

8 Résultats Obtenus

Les différents tests réalisés ont permis de vérifier :

- Le bon fonctionnement du serveur HTTP.
- La récupération correcte des données JSON.
- La gestion des timeouts.
- Le fonctionnement du mécanisme de `retry`.
- L'amélioration des performances grâce à `asyncio`.

9 Conclusion

Ce travail pratique a permis de découvrir les bases de la communication HTTP dans les systèmes distribués.

Les concepts étudiés, tels que le timeout, le `retry` et la programmation asynchrone, sont largement utilisés dans les architectures modernes afin d'améliorer la fiabilité et les performances des applications.